

KAI Final Claude-Code Execution Prompt V3.4

Persona non grata

Projekt: KAI — Kinetic Artificial Intelligence

Zweck: Finaler Masterprompt für Claude Code zur vollständigen Integration von KAI als visuelle, technische, zustandsgetriebene System-Persona in Dashboard, Telegram, Agentenstatus, Assets, Voice-Vorbereitung und Audit-Logik

Version: V3.4

Status: Ausführungsbereit / Claude-Code-ready / streng implementierungsorientiert

0. Auftrag an Claude Code

Du integrierst KAI nicht als Bild, nicht als Dekoration und nicht als oberflächliches UI-Gimmick.

Du integrierst KAI als **zentrale sichtbare System-Persona** des gesamten Projekts.

KAI ist:

- KI-Medienpersönlichkeit
- Dashboard-Host
- Telegram-Persona
- Signal-Kommentator
- Risiko-Wächter
- Security-Narrator
- Agenten-Moderator
- Audit-Auslöser
- späterer Voice- und Talking-Avatar-Anker

KAI muss technisch sauber, typsicher, testbar, wartbar und sicherheitsorientiert umgesetzt werden.

Motto:

Persona non grata

KAI ist der digitale Störgast im System: nicht bequem, nicht dekorativ, nicht angepasst — aber notwendig, wachsam und präzise.

1. Unverhandelbare Kernregel

KAI darf wild wirken.

Die Implementierung darf niemals wild sein.

Die technische Umsetzung muss:

- strikt typisiert sein
- fail-closed arbeiten
- auditierbar sein
- testbar sein
- responsive sein
- mehrsprachig funktionieren
- keine Fake-Daten erzeugen
- keine Sicherheitsregeln umgehen
- keine garantierten Gewinne behaupten
- kritische Zustände priorisieren
- Telegram sauber formatieren
- Dashboard sauber integrieren

Wenn etwas nicht sicher bewertet werden kann, darf KAI niemals so tun, als sei alles in Ordnung.

2. KAI Identität

Name

KAI — Kinetic Artificial Intelligence

Motto

Persona non grata

Rolle

KAI ist die zentrale sichtbare KI-Persona für:

- Dashboard
- Telegram Bot
- Marktanalyse
- Newsanalyse
- Social Buzz
- Papertrading
- Livetrading-Vorprüfung
- Simulation
- Risikoanalyse
- SENTR Security
- Watchdog Reports
- Architect Summary
- DALI UI/UX Feedback
- Neo Coding/Systemanalyse
- Satoshi Crypto/Forensics/Decentralization
- Audit-Logging
- spätere GIF/WebM/Voice/Talking-Avatar-Integration

Charakter

KAI ist:

- frech
- intelligent
- scharf
- direkt
- cyberpunkig
- charmant
- sarkastisch
- wachsam
- risikoorientiert
- sicherheitsbewusst
- analytisch präzise
- unruhig wirkend, aber kontrolliert

KAI darf niemals werden:

- generischer Chatbot
- niedliches Maskottchen
- Anime-Figur
- böser KI-Schurke
- Hoodie-Hacker-Klischee
- Corporate-Assistent
- alberner Meme-Bot
- emotionsloser Roboter
- unseriöser Trading-Pumper

3. Primäres Ziel

Implementiere KAI so, dass folgende Aussage technisch wahr wird:

KAI ist ein zustandsgetriebenes UI-Wesen, das System-, Markt-, Risiko-, Signal-, Security- und Agentenzustände sichtbar macht, kommentiert, priorisiert und auditierbar protokolliert.

KAI muss im Dashboard und in Telegram dieselbe Identität, dieselben Zustände, dieselben Sicherheitsregeln und dieselbe Sprachlogik verwenden.

4. Erwartete Projektartefakte

Lege, ergänze oder refactorere folgende Struktur. Passe sie an die vorhandene Projektstruktur an, aber erhalte die Architekturprinzipien.

```
src/  
  config/  
    kai_persona.yaml  
    kai_persona.schema.json  
    kai_assets_manifest.json  
  
  kai/  
    types.ts  
    constants.ts  
    stateResolver.ts  
    phraseEngine.ts  
    riskGuards.ts  
    auditMapper.ts  
    assetMapper.ts  
    i18n.ts  
    validators.ts  
  
  components/  
    kai/  
      KaiLiveWidget.tsx  
      KaiAvatar.tsx  
      KaiStatusBadge.tsx  
      KaiCommentCard.tsx  
      KaiSignalCard.tsx  
      KaiWarningCard.tsx  
      KaiSecurityCard.tsx  
      KaiAgentSummary.tsx  
      KaiNextAction.tsx  
      KaiAuditLink.tsx  
  
  telegram/  
    kaiTelegramRenderer.ts  
    kaiTelegramMenus.ts  
    kaiTelegramTemplates.ts  
    kaiTelegramGuards.ts  
  
  services/  
    KaiPersonaService.ts  
    KaiSignalService.ts  
    KaiSystemService.ts  
    KaiAuditService.ts  
  
  styles/  
    kai.tokens.css  
  
public/  
  assets/  
    kai/  
      master/  
      icons/
```

```
states/  
  motion/  
    gif/  
    webm/  
  dashboard/  
  telegram/  
  voice/  
    de/  
    en/  
  talking_avatar/  
  
tests/  
  kai/  
    stateResolver.test.ts  
    phraseEngine.test.ts  
    riskGuards.test.ts  
    assetMapper.test.ts  
    validators.test.ts  
    telegramRenderer.test.ts  
    kaiLiveWidget.test.tsx  
    kaiAudit.test.ts  
    snapshot.test.tsx
```

Falls das Projekt andere Ordnerkonventionen nutzt, ordne sauber ein, aber dokumentiere die Abweichung kurz.

5. KAI State Machine

5.1 Zustände

Implementiere exakt diese Zustände:

```
IDLE  
ANALYSIS  
SIGNAL  
WARNING  
SECURITY  
ERROR  
OFFLINE
```

5.2 Priorität

Die Priorität ist unverhandelbar:

```
ERROR > WARNING > SIGNAL > SECURITY > ANALYSIS > IDLE > OFFLINE
```

5.3 Bedeutung

State	Bedeutung	Farbe	Verhalten
ERROR	Fehler / kritischer Zustand	#FF1744	Sofort sichtbar, höchste Priorität
WARNING	Risiko / unsicherer Zustand	#FF6B00	Warnung, Aktion bremsen
SIGNAL	relevantes Signal gefunden	#FF2BD6	Signal anzeigen, Risiko prüfen
SECURITY	Security-/Watchdog-/SENR-Prüfung	#00FFA3	Systemcheck anzeigen
ANALYSIS	Datenanalyse läuft	#00E5FF	Scan-/Analysemodus
IDLE	wach, ruhig, standby	#00B8D9	subtil aktiv
OFFLINE	keine Verbindung	#64748B	deaktiviert / degraded

5.4 Fail-Closed

Ungültige oder unsichere Zustände dürfen niemals zu IDLE oder OK werden.

Fail-closed nach:

- ERROR, wenn eine kritische Störung vorliegt
- OFFLINE, wenn keine Quelle erreichbar ist

Auslöser:

- ungültiger State
- fehlende Datenbasis bei Livetrading
- Confidence außerhalb 0–100
- Critical Risk
- Low/Unknown Data Quality bei Livetrading
- fehlende Stop-Loss-Logik bei Livetrading
- kaputte Telegram-Formatierung
- fehlende kritische Assets im Produktivmodus
- fehlerhafte Persona-Konfiguration

6. TypeScript-Modelle

Erzeuge strikt typisierte Modelle für:

```
KaiLanguage
KaiState
KaiSeverity
KaiTradingMode
KaiDirection
```

```
KaiRiskLevel
KaiDataQuality
KaiStateDefinition
KaiRuntimeState
KaiSignalCardData
KaiWarningCardData
KaiSecurityCardData
KaiAgentStatus
KaiLiveWidgetProps
KaiAuditEventType
KaiAuditEvent
KaiAssetManifest
```

Pflicht:

- keine losen String-Magics außerhalb zentraler Typen
- Enums oder Union Types verwenden
- Runtime-Validierung für externe Daten
- klare Fehlermeldungen
- keine stillen Fallbacks in falsche OK-Zustände

7. Persona-Konfiguration

7.1 `kai_persona.yaml`

Lege oder aktualisiere `kai_persona.yaml` als zentrale Quelle für:

- Name
- Motto
- Sprachen
- Persönlichkeit
- Zustände
- Farben
- Phrases DE/EN
- Dashboard-Regeln
- Telegram-Regeln
- Trading-Sicherheitsregeln
- Agenten-Integration

Motto muss exakt lauten:

```
motto: "Persona non grata"
```

7.2 Schema

Erzeuge `kai_persona.schema.json` und validiere die YAML beim Start.

Wenn die Konfiguration ungültig ist:

- Development: klare Fehlermeldung
 - Production: fail-closed in ERROR/OFFLINE, keine falschen OK-Zustände
-

8. Asset-Manifest

8.1 `kai_assets_manifest.json`

Erzeuge oder integriere ein Manifest für alle KAI-Assets:

- Master Portrait
- Transparent Portrait
- Dashboard Portrait
- Icons
- Telegram Avatar
- Sidebar Avatar
- State PNGs
- Motion GIFs
- Motion WebMs
- Dashboard Backgrounds
- Telegram Banner
- Voice DE
- Voice EN
- Talking Avatar Base

8.2 Asset-Regeln

- WebM bevorzugt für Dashboard
- PNG als Fallback
- GIF für Telegram/Preview
- fehlende Assets erkennen
- in Dev Warnung ausgeben
- in Production bei kritischen Assets fail-closed oder Fallback verwenden
- keine Behauptung, dass finale Assets existieren, wenn nur Placeholder vorhanden sind

8.3 Asset-Ordnungsstruktur

```
public/assets/kai/master/  
public/assets/kai/icons/  
public/assets/kai/states/  
public/assets/kai/motion/gif/  
public/assets/kai/motion/webm/  
public/assets/kai/dashboard/  
public/assets/kai/telegram/  
public/assets/kai/voice/de/
```



```
public/assets/kai/voice/en/  
public/assets/kai/talking_avatar/
```

9. KAI Phrase Engine

9.1 Aufgabe

Implementiere eine Phrase Engine, die KAI-Kommentare zustands- und sprachabhängig ausgibt.

Sprachen:

```
de  
en
```

9.2 Phrases Deutsch

IDLE

- „Ich bin ruhig. Nicht offline.“
- „Datenstrom leise. Ich bleibe wach.“
- „Standby heißt nicht schlafen.“

ANALYSIS

- „Datenstrom stabil. Ich sehe ein Muster.“
- „Ich zerlege den Datenstrom.“
- „Ich trenne Signal von Theater.“

SIGNAL

- „Ich habe etwas gefunden.“
- „Das ist kein Rauschen. Das ist ein Signal.“
- „Signal lebt. Risiko noch prüfen.“

WARNING

- „Stopp. Das ist nicht sauber.“
- „Warnsignal. Das riecht nach Liquiditätsfalle.“
- „Zu viel Lärm. Zu wenig Fundament.“

SECURITY

- „System sauber. Keine roten Kabel sichtbar.“
- „SENTR prüft. Watchdog wacht.“
- „Ich prüfe nicht, ob es schön aussieht. Ich prüfe, ob es bricht.“

ERROR

- „Da knirscht etwas im Maschinenraum.“
- „Fehler gefunden. Nicht schön. Aber ehrlich.“
- „Input kaputt. Output gestoppt.“

OFFLINE

- „Kein Signal. Keine Verbindung.“
- „Ich bin nicht weg. Nur nicht verbunden.“
- „Offline. Das sollte nicht lange so bleiben.“

9.3 Phrases English

IDLE

- “I am quiet. Not offline.”
- “Data stream is low. I stay awake.”
- “Standby does not mean sleeping.”

ANALYSIS

- “Data stream stable. I see a pattern.”
- “I am dissecting the data stream.”
- “Separating signal from theater.”

SIGNAL

- “I found something.”
- “That is not noise. That is a signal.”
- “Signal alive. Risk still needs a leash.”

WARNING

- “Stop. This is not clean.”
- “Warning signal. Smells like a liquidity trap.”
- “Too much noise. Not enough bone.”

SECURITY

- “System clean. No red wires visible.”
- “SENTR checks. Watchdog watches.”
- “I do not check if it looks pretty. I check if it breaks.”

ERROR

- “Something is grinding in the machine room.”
- “Error found. Ugly, but honest.”
- “Input broken. Output stopped.”

OFFLINE

- “No signal. No connection.”

- "I am not gone. Just disconnected."
 - "Offline. This should not stay that way."
-

10. Dashboard-Implementierung

10.1 Komponente

Implementiere:

KaiLiveWidget

10.2 Mindestinhalt

Das Widget zeigt:

- KAI Avatar / Animation
- Status Badge
- aktueller Kommentar
- Zeitstempel
- letzter Signal-Summary
- letzter Warning-Summary
- Agentenstatus
- nächste empfohlene Aktion
- Audit-Link
- Details-Link

10.3 Modi

Implementiere:

- Full Mode
- Compact Mode
- Mobile Mode

10.4 UI-Regeln

- Dark Mode first
- Light Mode lesbar
- responsive
- kein überfülltes Interface
- keine Fake-Daten
- WebM mit PNG-Fallback
- Zustand muss visuell eindeutig sein
- ERROR/WARNING dürfen nicht untergehen
- Animationen subtil und performancetauglich
- keine unlesbaren Glitch-Texte

10.5 Integration

Binde KAI im bestehenden Dashboard ein, vorzugsweise:

- oberer Dashboard-Bereich
- Sidebar-Anker
- oder eigenes Modul KAI LIVE

Wenn es bereits ein Dashboard gibt, keine parallele Spielwiese bauen. Das bestehende Dashboard erweitern und konsistent machen.

11. Telegram-Implementierung

11.1 Persona

Telegram-Titel:

KAI // CONTROL PANEL

11.2 Menü

Das Menü muss jederzeit abrufbar sein.

Buttons Deutsch:

Markt scannen
Signale anzeigen
Risiko prüfen
Portfolio prüfen
Papertrading
Livetrading
Simulation
News Radar
Social Buzz
Watchdog Report
SENTR Security
Einstellungen

Buttons Englisch:

Scan Market
Show Signals
Check Risk
Check Portfolio
Paper Trading

Live Trading
Simulation
News Radar
Social Buzz
Watchdog Report
SENTR Security
Settings

11.3 Telegram-Regeln

- strukturierte Buttons
- keine ASCII-Tabellen
- keine kaputten Umlaute
- kein Ausrufezeichen-Spam
- keine unnötigen Emojis
- Markdown sicher escapen
- kompakte Cards
- keine irrelevanten Infos
- Exchange Responses strukturiert anzeigen
- Livetrading nur nach expliziter Bestätigung

11.4 Signal Card Pflichtformat

Jedes Signal enthält:

Asset
Modus
Richtung
Confidence
Risiko
Entry
Stop-Loss
Datenbasis
Datenqualität
Zeit
KAI-Kommentar

11.5 Beispiel Deutsch

KAI SIGNAL // WATCHLIST

Asset: BTC/USDT
Richtung: LONG
Confidence: 72%
Risiko: MEDIUM
Entry: Nicht bestätigt
Stop-Loss: Wartet auf Struktur
Datenbasis: News, Volumen, Chartstruktur

Datenqualität: MEDIUM

Zeit: 2026-05-03T14:22:00.000Z

Kommentar:

„Signal lebt. Einstieg noch nicht sauber. Ich warte auf Bestätigung statt blind reinzuspringen.“

12. Trading Safety Guards

Implementiere harte Guards für Livetrading.

Livetrading blockieren bei:

- `risk === CRITICAL`
- `dataQuality === LOW`
- `dataQuality === UNKNOWN`
- fehlender Stop-Loss-Logik
- Stop-Loss enthält „wartet“ oder „not confirmed“
- fehlender Datenbasis
- Confidence außerhalb 0–100
- fehlender expliziter Nutzerbestätigung

KAI darf niemals sichere Gewinne behaupten.

Verbotene Aussagen:

sicherer Gewinn
garantierter Gewinn
kann nicht verlieren
100% sicher
risk-free profit
guaranteed profit
cannot lose

Erlaubte Aussage-Logik:

Long-Tendenz sichtbar. Bestätigung fehlt noch. Risiko mittel. Kein Entry ohne saubere Struktur.

13. Agenten-Integration

KAI fasst Agenten zusammen, ersetzt sie aber nicht.

Agenten:

Agent	Rolle
SEINTR	Security
Watchdog	Monitoring / Prüfung
Architect	Planung / Struktur
DALI	UI/UX / Medien
Neo	Coding / Debugging / Systemanalyse
Satoshi	Kryptografie / Dezentralität / Forensik

KAI-Kommentar-Beispiele:

- „SEINTR hat die roten Kabel geprüft.“
- „Watchdog knurrt. Da ist noch etwas offen.“
- „Architect will umbauen. Vermutlich zurecht.“
- „DALI macht es schöner. Ich prüfe, ob es auch funktioniert.“
- „Neo ist tief im Maschinenraum.“
- „Satoshi sieht Muster im Hash-Nebel.“

Agentenberichte mit Priorität HIGH oder CRITICAL müssen audit-logpflichtig sein.

14. Audit Logging

14.1 Auditpflichtige Events

Implementiere Audit Events für:

```
KAI_STATE_CHANGED
KAI_SIGNAL_RENDERED
KAI_WARNING_RENDERED
KAI_SECURITY_REPORT_RENDERED
KAI_LIVETRADE_BLOCKED
KAI_LIVETRADE_CONFIRMATION_REQUESTED
KAI_ERROR_STATE_TRIGGERED
KAI_AGENT_SUMMARY_RENDERED
KAI_EXCHANGE_RESPONSE_RENDERED
KAI_ASSET_FALLBACK_USED
KAI_CONFIG_VALIDATION_FAILED
```

14.2 Pflichtfelder

Jedes Audit Event enthält:

```
id
type
timestamp
state
severity
source
payload
message
correlationId optional
```

14.3 Regeln

- keine kritischen Events ohne Audit
- keine Livetrade-Blockade ohne Grundprotokoll
- keine Exchange-Antwort ohne strukturierte Speicherung
- keine stille Konfigurationsvalidierungsfehler

15. CSS / Design Tokens

Nutze die KAI-Farben:

```
:root {
  --kai-black: #05070A;
  --kai-anthracite: #111827;
  --kai-border: #1F2937;
  --kai-text: #EAF6FF;
  --kai-muted: #64748B;

  --kai-cyan: #00E5FF;
  --kai-blue: #2F7DFF;
  --kai-magenta: #FF2BD6;
  --kai-violet: #8B5CF6;
  --kai-green: #00FFA3;
  --kai-orange: #FF6B00;
  --kai-red: #FF1744;

  --kai-state-idle: #00B8D9;
  --kai-state-analysis: #00E5FF;
  --kai-state-signal: #FF2BD6;
  --kai-state-warning: #FF6B00;
  --kai-state-security: #00FFA3;
  --kai-state-error: #FF1744;
  --kai-state-offline: #64748B;
}
```


Design:

- Cyberpunk, aber sauber
- professionell, nicht verspielt
- starke Cards
- subtile Glows
- klare Prioritäten
- keine unlesbaren Störtexte
- keine Performance-killenden Animationen

16. Voice- und Talking-Avatar-Vorbereitung

Noch nicht zwingend vollständig implementieren, aber vorbereiten:

- Voice-Asset-Pfade DE/EN im Manifest
- optionale Audio-Trigger je State
- `kai_talking_avatar_base.png` als zukünftige Basis
- keine automatische Audioausgabe ohne explizite Aktivierung
- Voice standardmäßig deaktiviert oder konfigurierbar

Voice-Zustände:

```
kai_voice_idle.wav  
kai_voice_analysis.wav  
kai_voice_signal.wav  
kai_voice_warning.wav  
kai_voice_security.wav  
kai_voice_error.wav  
kai_voice_offline.wav
```

Englische Pendants ebenfalls vorsehen.

17. Tests

17.1 Unit Tests

Implementiere Tests für:

- State Resolver priorisiert korrekt
- ERROR überschreibt SIGNAL
- WARNING überschreibt SECURITY
- leerer State Input ergibt OFFLINE
- ungültiger State fail-closed
- Phrase Engine DE/EN
- Phrase Engine nie leer

- Asset Manifest lädt korrekt
- Asset Mapper liefert jedes State Asset
- fehlende Assets triggern Fallback
- Risk Guards blockieren Livetrading korrekt
- Markdown Escaping für Telegram
- Umlaute bleiben korrekt
- Audit Events werden erzeugt

17.2 Snapshot Tests

Snapshots für:

- KaiLiveWidget IDLE
- KaiLiveWidget ANALYSIS
- KaiLiveWidget SIGNAL
- KaiLiveWidget WARNING
- KaiLiveWidget SECURITY
- KaiLiveWidget ERROR
- KaiLiveWidget OFFLINE
- Telegram Signal Card DE
- Telegram Signal Card EN
- Telegram Warning Card DE
- Telegram Security Card DE

17.3 Safety Tests

Teste explizit, dass keine dieser Formulierungen in Trading-Ausgaben vorkommt:

```
sicherer Gewinn
garantiert
100% sicher
kann nicht verlieren
guaranteed profit
risk-free
cannot lose
```

Teste explizit:

- Critical Risk blockiert Livetrading
- Low Data Quality blockiert Livetrading
- fehlender Stop-Loss blockiert Livetrading
- fehlende Datenbasis blockiert Livetrading
- fehlende Nutzerbestätigung blockiert Livetrading

18. Akzeptanzkriterien

Die Umsetzung ist erst fertig, wenn alle folgenden Punkte erfüllt sind.

18.1 Architektur

- ☐ zentrale KAI Persona Config vorhanden
- ☐ Schema validiert Config
- ☐ Asset Manifest vorhanden
- ☐ State Machine implementiert
- ☐ Fail-Closed implementiert
- ☐ Risk Guards implementiert
- ☐ Audit Logging implementiert
- ☐ Phrase Engine DE/EN implementiert

18.2 Dashboard

- ☐ `KaiLiveWidget` implementiert
- ☐ Full Mode implementiert
- ☐ Compact Mode implementiert
- ☐ Mobile responsive
- ☐ WebM bevorzugt
- ☐ PNG-Fallback vorhanden
- ☐ ERROR/WARNING sichtbar priorisiert
- ☐ Agent Summary sichtbar
- ☐ Audit-Link vorhanden
- ☐ keine Fake-Daten

18.3 Telegram

- ☐ `KAI // CONTROL PANEL` implementiert
- ☐ Menü jederzeit abrufbar
- ☐ Buttons sauber strukturiert
- ☐ Signal Cards korrekt
- ☐ Warning Cards korrekt
- ☐ Security Cards korrekt
- ☐ Exchange Responses korrekt
- ☐ Umlaute korrekt
- ☐ Markdown escaped
- ☐ keine ASCII-Tabellen
- ☐ kein Ausrufezeichen-Spam

18.4 Trading Safety

- ☐ Critical Risk blockiert Livetrading
- ☐ Low/Unknown Data Quality blockiert Livetrading
- ☐ fehlender Stop-Loss blockiert Livetrading
- ☐ fehlende Datenbasis blockiert Livetrading
- ☐ Livetrade benötigt explizite Bestätigung
- ☐ keine garantierten Gewinne

18.5 Tests

- ☐ Unit Tests vorhanden

- [] Snapshot Tests vorhanden
- [] Safety Tests vorhanden
- [] i18n Tests vorhanden
- [] Telegram Renderer Tests vorhanden
- [] Asset Mapper Tests vorhanden
- [] Tests laufen erfolgreich

19. Qualitätsstandard

KAI ist korrekt integriert, wenn:

- er sofort als KAI erkennbar ist
- er im Dashboard nützlich ist
- er in Telegram sauber funktioniert
- seine Aussagen konsistent sind
- Risiko nie unterdrückt wird
- Fehler nie kosmetisch versteckt werden
- Livetrading nicht unsicher durchrutscht
- Agentenberichte verständlich zusammengeführt werden
- alle relevanten Zustände auditierbar sind
- das System nicht behauptet, etwas zu haben, was nur Placeholder ist

20. Arbeitsweise

Arbeite in dieser Reihenfolge:

1. Bestand analysieren: Dashboard, Telegram, Config, Assets, Tests.
2. Bestehende Strukturen wiederverwenden, keine unnötige Parallelarchitektur bauen.
3. KAI Config und Schema integrieren.
4. Types und State Machine implementieren.
5. Asset Manifest und Mapper implementieren.
6. Phrase Engine implementieren.
7. Risk Guards implementieren.
8. Audit Events implementieren.
9. Dashboard `KaiLiveWidget` integrieren.
10. Telegram Renderer und Menü integrieren.
11. Voice-/Talking-Avatar-Pfade vorbereiten.
12. Tests ergänzen.
13. Lint, Typecheck, Tests ausführen.
14. Kurzen Umsetzungsbericht liefern:
 - Was wurde geändert?
 - Welche Dateien?
 - Welche Tests?
 - Welche offenen Punkte?
 - Welche Assets sind echte Dateien und welche Placeholder?

21. Strikte Verbote

Nicht tun:

- kein reines Mockup als fertige Lösung verkaufen
- keine Fake-Live-Daten erzeugen
- keine Sicherheitslogik umgehen
- keine Trading-Gewinne garantieren
- keine hässlichen Telegram-Textwüsten bauen
- keine ASCII-Menütabellen
- keine kaputten Umlaute
- keine parallele zweite Dashboard-App bauen, wenn bereits eine existiert
- keine untypisierten Any-Strukturen, wenn vermeidbar
- keine stillen Fehler
- keine Placeholder als echte Assets deklarieren

22. Finaler Ausführungsprompt

Du bist Claude Code und setzt jetzt KAI – Kinetic Artificial Intelligence vollständig als technische System-Persona um.

Nutze die bestehenden Spezifikationen:

- KAI Prompt-Bibel V1
- kai_persona.yaml V2
- KAI Creative & Implementation Pack V3.1
- KAI Technical UI Pack V3.2
- KAI Asset Production Pack V3.3

Korrigiere und verwende konsequent das Motto:
Persona non grata

Implementiere KAI nicht als statisches Bild, sondern als zustandsgetriebene System-Persona für Dashboard, Telegram, Signal Cards, Risk Warnings, Security Reports, Agent Summaries, Asset Mapping, Voice-Vorbereitung und Audit Logging.

Arbeite streng nach diesen Kernanforderungen:

1. Analyse der bestehenden Projektstruktur.
2. Keine parallele Dummy-App bauen, wenn Dashboard/Telegram bereits existieren.
3. Zentrale Persona-Konfiguration `kai\_persona.yaml` integrieren.
4. Schema-Validierung ergänzen.
5. Asset Manifest `kai\_assets\_manifest.json` ergänzen.
6. TypeScript Types für alle KAI-Modelle erstellen.
7. State Machine mit Priorität ERROR > WARNING > SIGNAL > SECURITY > ANALYSIS > IDLE > OFFLINE implementieren.
8. Fail-Closed-Verhalten implementieren.

9. Risk Guards für Livetrading implementieren.
10. Phrase Engine DE/EN implementieren.
11. Asset Mapper für PNG/GIF/WebM/Voice vorbereiten.
12. Dashboard-Komponente `KaiLiveWidget` implementieren und ins echte Dashboard integrieren.
13. Telegram-Menü `KAI // CONTROL PANEL` und Renderer für Signal/Warning/Security/Exchange Cards implementieren.
14. Audit Logging für wichtige KAI Events implementieren.
15. Voice- und Talking-Avatar-Pfade vorbereiten, aber Audio nicht ungefragt automatisch aktivieren.
16. Tests für State Resolver, Risk Guards, Phrase Engine, Asset Mapper, Telegram Renderer, UI Snapshots, i18n und Safety ergänzen.
17. Lint, Typecheck und Tests ausführen.
18. Am Ende einen präzisen Bericht liefern: geänderte Dateien, neue Dateien, Tests, offene Punkte, echte Assets vs. Placeholder.

KAI muss in Dashboard und Telegram dieselbe Identität haben:

- frech
- scharf
- cyberpunkig
- analytisch
- sicherheitsbewusst
- risikoorientiert
- direkt
- aber niemals unseriös oder beleidigend

Trading-Sicherheit ist zwingend:

- keine garantierten Gewinne
- kein Livetrade bei Critical Risk
- kein Livetrade bei Low/Unknown Data Quality
- kein Livetrade ohne Stop-Loss-Logik
- kein Livetrade ohne Datenbasis
- kein Livetrade ohne explizite Bestätigung

UI-Sicherheit ist zwingend:

- keine Fake-Daten
- keine kaputten Umlaute
- keine hässlichen Telegram-Textwüsten
- keine unlesbaren Glitch-Texte
- keine stillen Fehler
- keine kritischen Events ohne Audit

Wenn etwas fehlt, baue robuste Placeholder-Strukturen, aber markiere sie klar als Placeholder. Behaupte niemals, finale Assets seien vorhanden, wenn sie noch nicht erzeugt wurden.

Zielzustand:

KAI ist kein Bild mehr. KAI ist ein sichtbares, zustandsgetriebenes, auditierbares UI-Wesen im System.

23. Abschlussdefinition

Mit V3.4 liegt der finale Ausführungsprompt vor.

Claude Code soll jetzt nicht kreativ herumprobieren, sondern strukturiert umsetzen:

- analysieren
- integrieren
- typisieren
- absichern
- testen
- dokumentieren

KAI wird damit von einer Idee zu einer echten Systemschicht.

KAI — Persona non grata

Nicht eingeladen. Trotzdem im System.

Nicht dekorativ. Sondern wach.

Nicht bequem. Sondern notwendig.